

Laboratory Report of Nanjing University of Information Science and Technology

Experiment Title: Cloud-Native Automated Deployment

Class: Cloud Computing

Name: Lu Weichi

ID: 202383910009

I. Experimental Purpose

The primary goal of this experiment is to introduce core concepts of cloud computing through hands-on practice with Platform-as-a-Service (PaaS) and automation. By completing this lab, students will learn how to:

Use GitHub as a cloud-based code hosting and collaboration platform.

Deploy a static website using GitHub Pages powered by GitHub Actions (CI/CD).

Trigger automatic cloud builds by modifying files in a repository.

Customize a web application's UI and content in a cloud-native way.

II. Experiment Contents

This lab consists of five integrated modules simulating a professional cloud deployment lifecycle:

Resource acquisition (forking a repository)

Enabling hosting service (GitHub Pages + Actions)

Triggering cloud automation (modifying HTML and watching the pipeline)

Verification and global access (obtaining the live URL)

Content personalization and multimedia integration (CSS changes and adding an image)

III. Detailed Steps of the Experiment

Note: All screenshots must include visible system time, desktop environment, or any other personal identifier to prove they were taken on your own computer.

Step 1: Resource Acquisition (Fork the Repository)

Action:

Log in to your GitHub account.

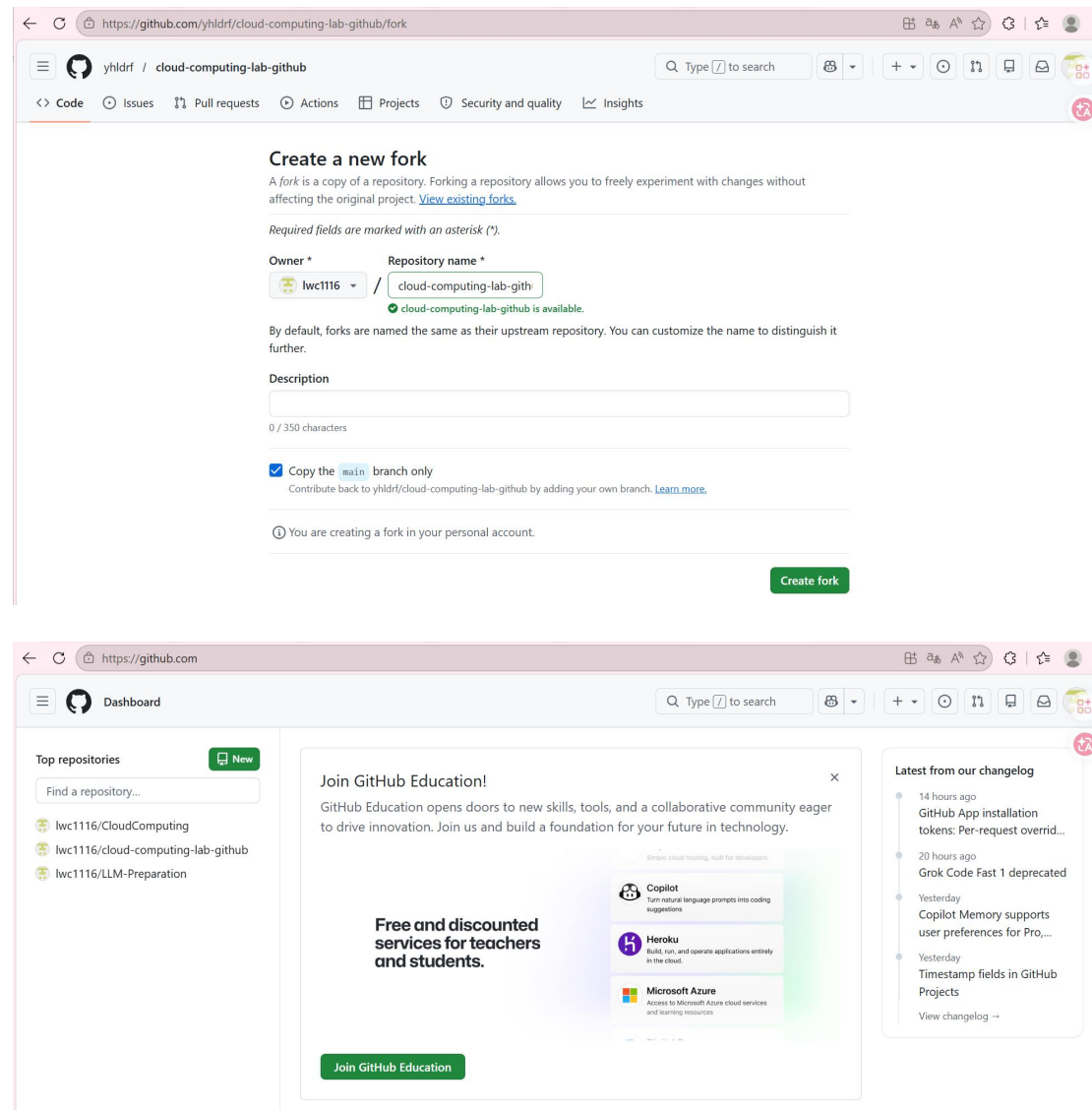
Navigate to the source repository:

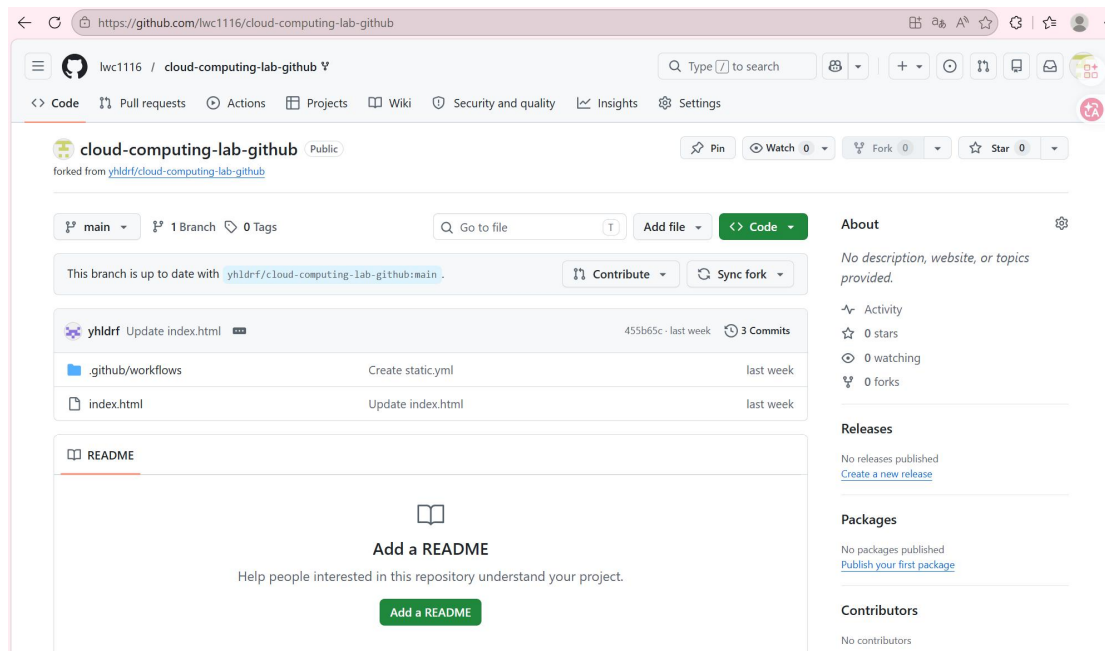
<https://github.com/yhldrf/cloud-computing-lab-github>

Click the **Fork** button (top-right corner).

Select your own account as the destination and click **Create fork**.

Screenshot 1 – Forked repository





Step 2: Enabling the Hosting Service (GitHub Pages with Actions)

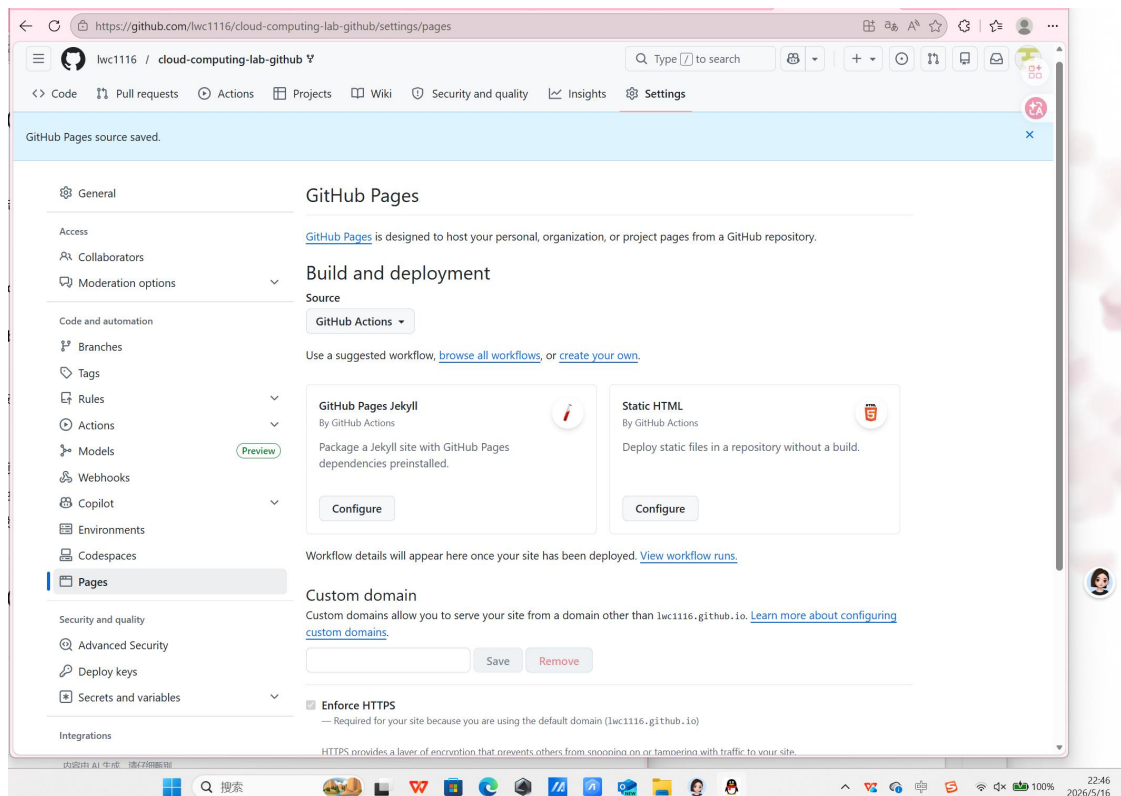
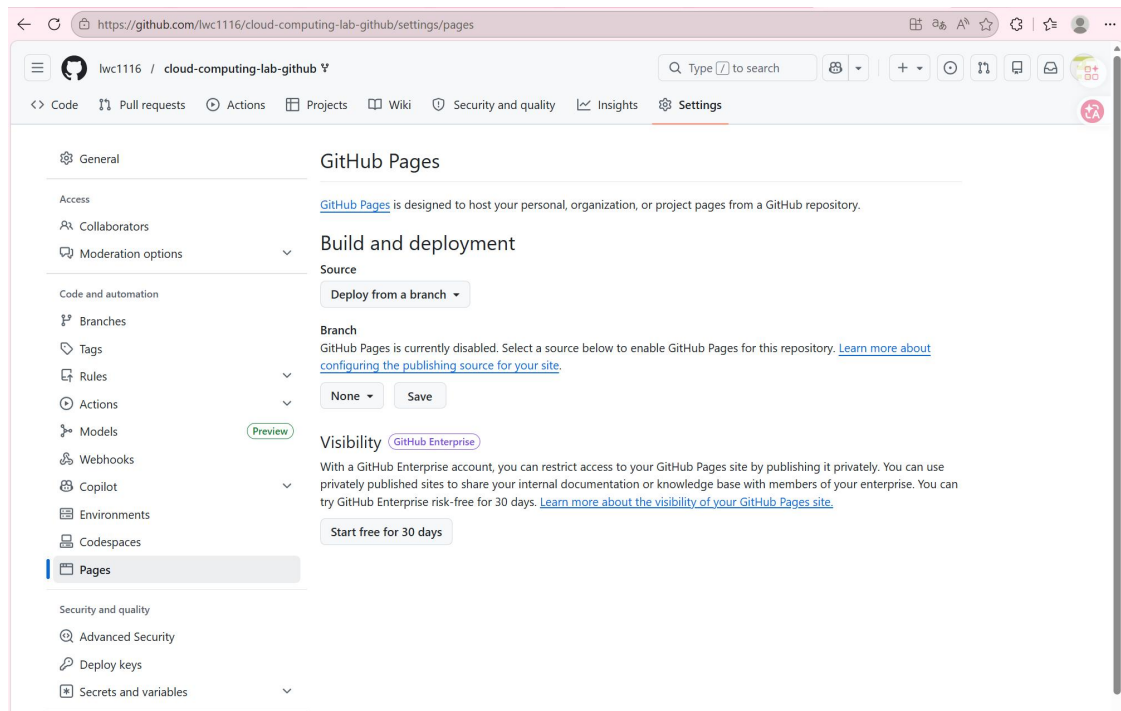
Action:

In your forked repository, go to **Settings** → **Pages** (under “Code and automation”).

Under “Build and deployment” → “Source”, select **GitHub Actions**.

No further action is needed – the setting saves automatically.

Screenshot 2 – GitHub Pages source set to Actions



Step 3: Triggering Cloud Automation

3.1 Modify index.html (Change your name)

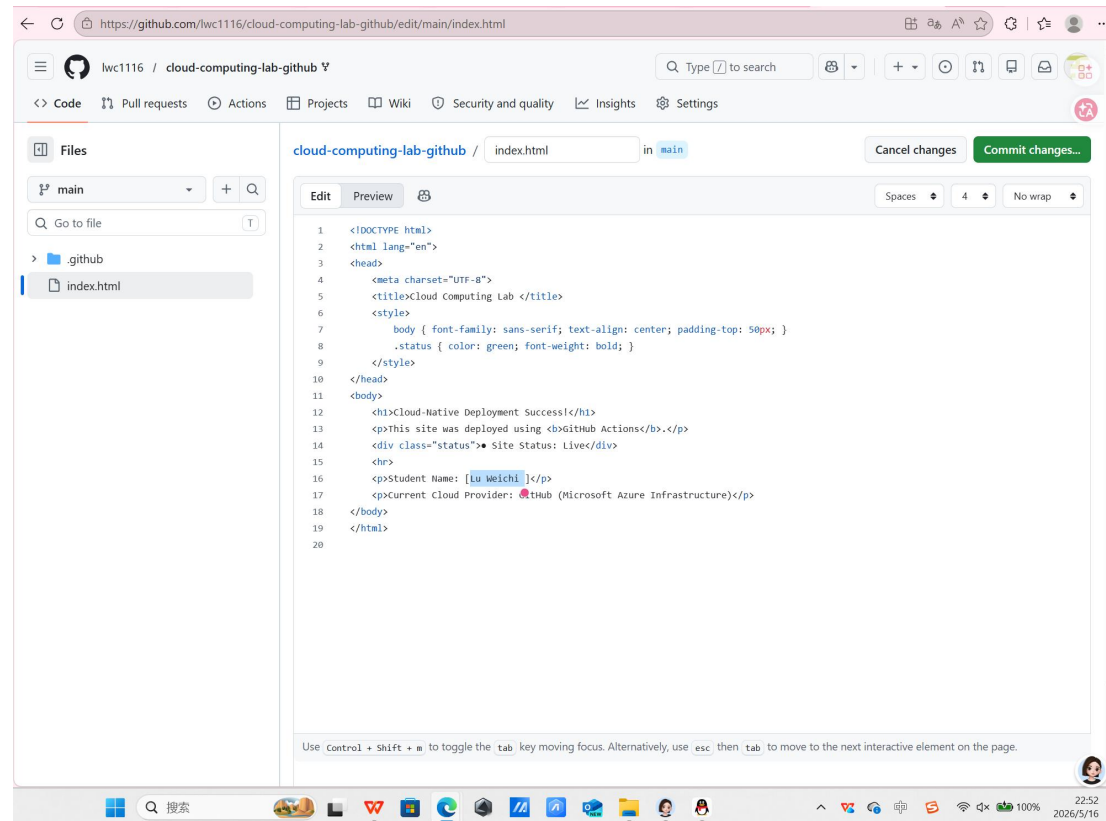
Action:

Go back to the **Code** tab of your repository.

Click on `index.html` and then the pencil icon (“Edit this file”).

Locate the placeholder `[Enter Your Name Here]` and replace it with your real name.

Screenshot 3 – Editing name in index.html



Action:

4. Scroll down, enter a commit message (e.g., “Update my name”), keep “Commit directly to the main branch” selected, and click **Commit changes**.

3.2 Enable GitHub Actions (if disabled for forked repo)

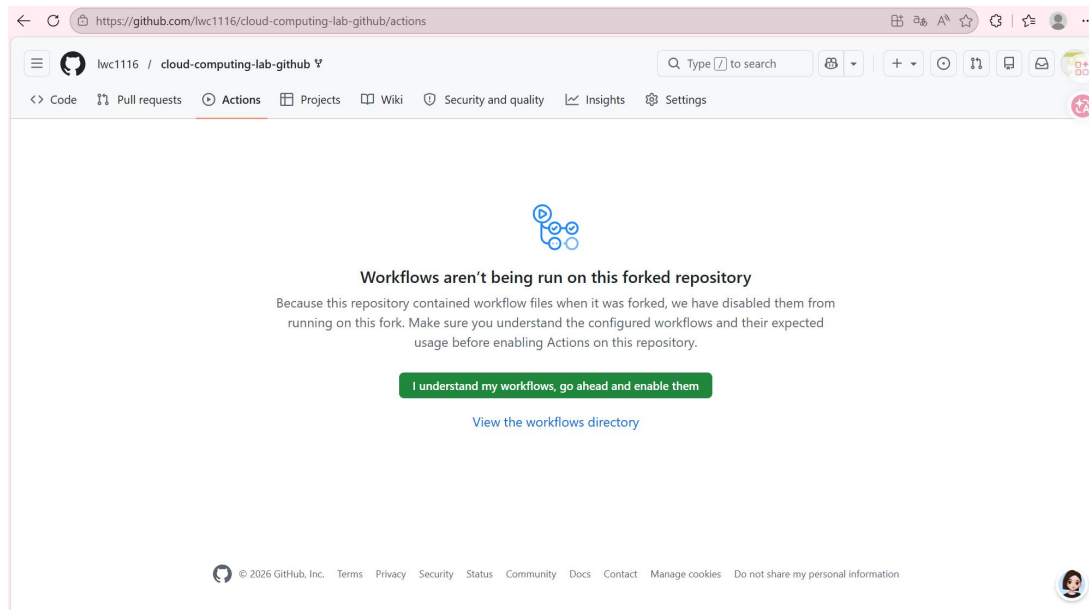
Important: When you first click the **Actions** tab, GitHub may show a warning:

“Workflows aren't being run on this forked repository... because this repository contained workflow files when it was forked.”

Solution:

Click the blue button “**I understand my workflows, go ahead and enable them**”.

Screenshot 4 – Enabling Actions manually



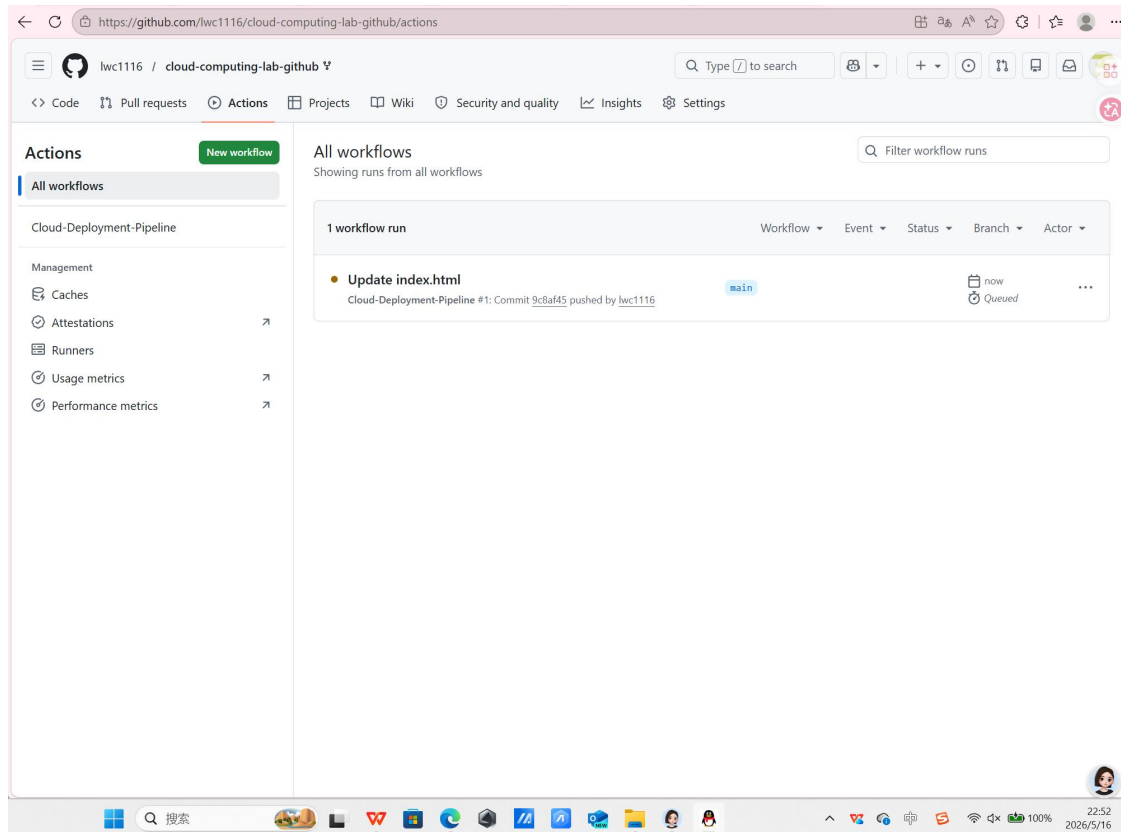
3.3 Watch the workflow run

Action:

After enabling Actions, you need to trigger a new workflow. Make a small extra change to `index.html` (e.g., add a space or change the name again) and commit again.

Then click the **Actions** tab. You will see a workflow named `Cloud-Deployment-Pipeline` running (yellow circle).

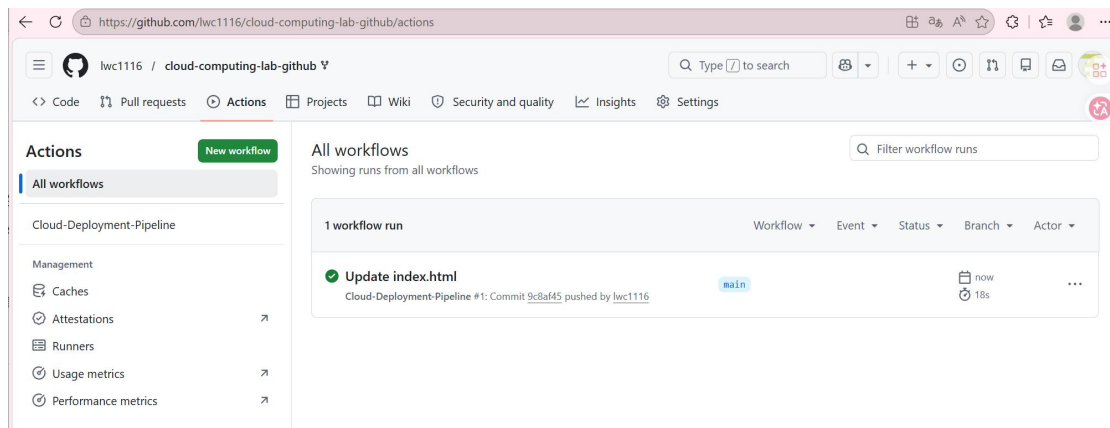
Screenshot 5 – Workflow in progress



Action:

Wait about 1–2 minutes until the circle turns green (✓ success).

Screenshot 6 – Workflow completed successfully



Step 4: Verification & Global Access

Action:

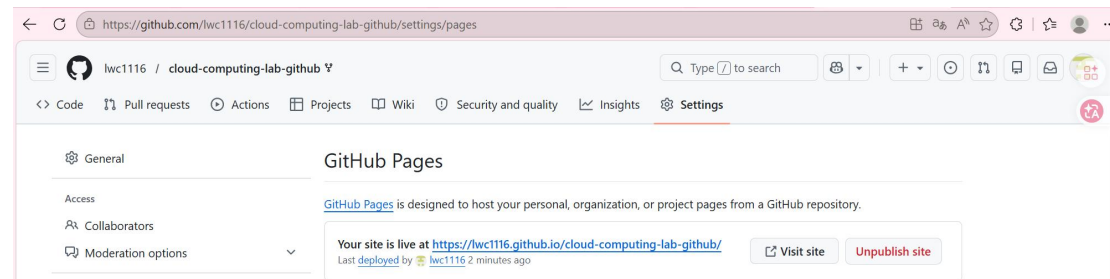
Go back to **Settings** → **Pages**.

Find the live URL:

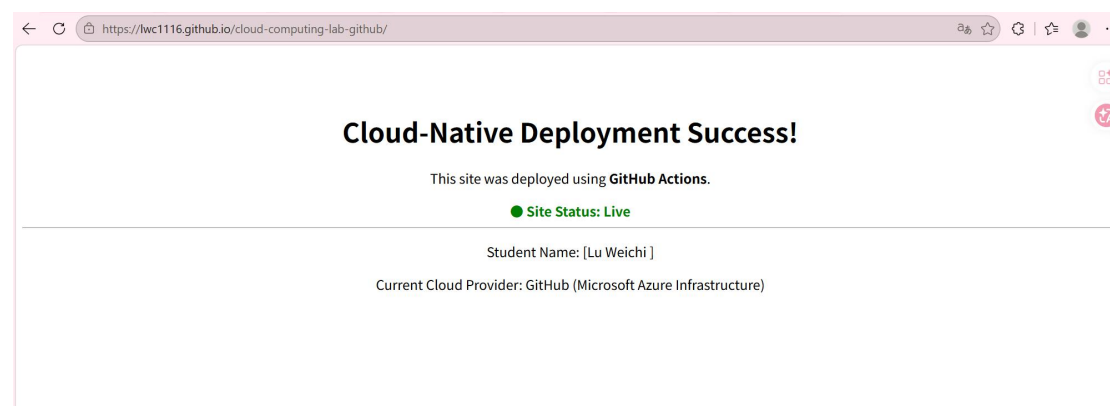
`https://your-username.github.io/cloud-computing-lab-github/`

Open this URL in a new browser tab.

Screenshot 7 – Live URL in Pages settings



Screenshot 8 – Website successfully deployed



Step 5: Content Personalization & Multimedia Integration (Optional but recommended)

5.1 Modify Theme Color (CSS)

Action:

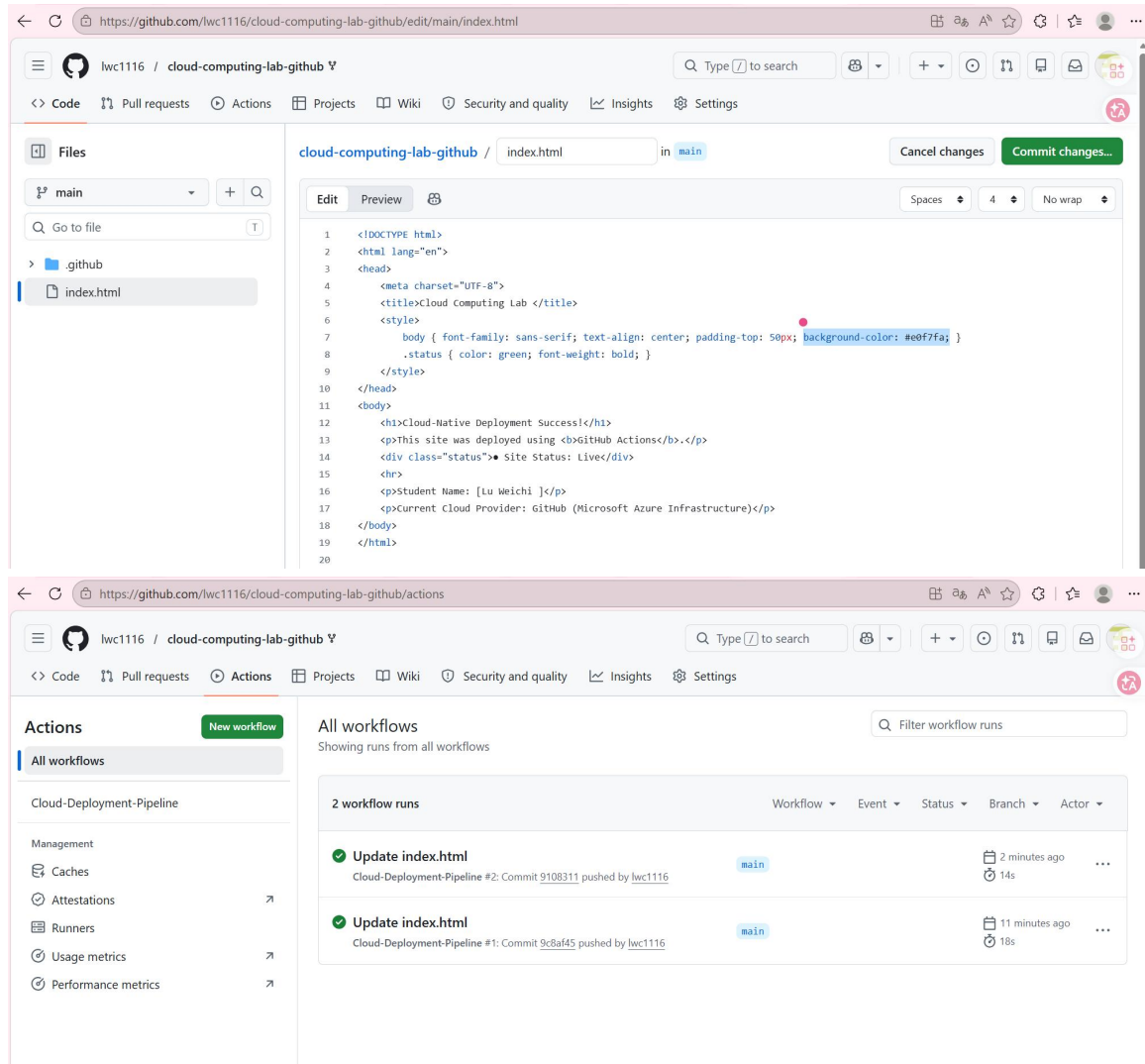
Edit `index.html` again (pencil icon).

Inside the `<style>` block, add a `background-color` property to the `body` rule. For example:

```
body { font-family: sans-serif; text-align: center; padding-top: 50px; background-color: #e0f7fa; /* new light cyan background */ }
```

Commit the change.

Screenshot 9 – Adding background-color in CSS

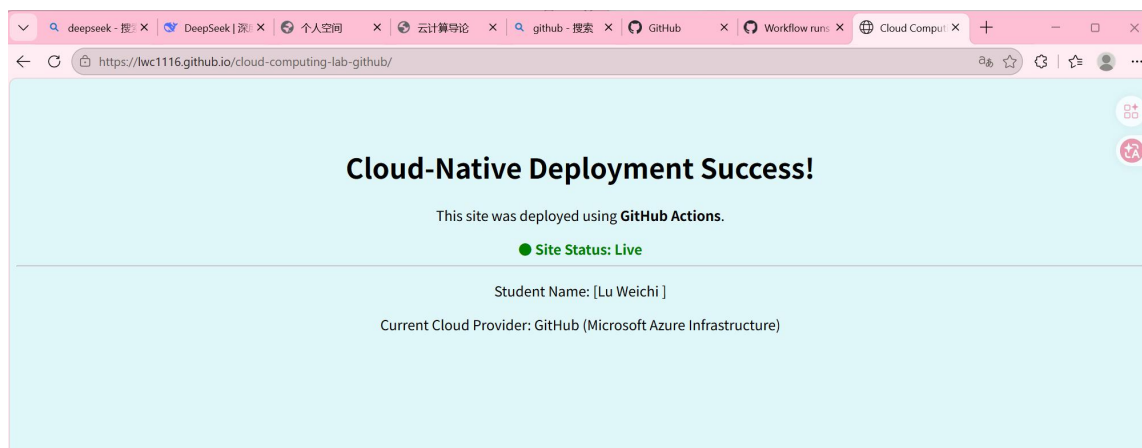


Action:

After committing, go to **Actions** and watch the automatic new build (continuous deployment).

Refresh your live website – the background should change to light cyan.

Screenshot 10 – Website with new background color



5.2 Multimedia Integration (Add an Image)

Action:

Edit `index.html` again.

Inside the `<body>` (e.g., after the status `<div>` but before the `<hr>`), add:

```

```

Commit the change.

Screenshot 11 – Adding img tag in the HTML

The first screenshot shows the GitHub web interface for the repository `lwc1116 / cloud-computing-lab-github`. The file `index.html` is open, showing the HTML code. The `<img>` tag has been added to the `<body>` section, as indicated by the blue highlight on line 15. The code includes a meta charset, title, and CSS styles for the page layout.

The second screenshot shows the `Actions` tab in the same repository. It displays a table of workflow runs for the `Cloud-Deployment-Pipeline`. The table has columns for Workflow, Event, Status, Branch, and Actor. Three runs are listed, all with a status of 'Completed' (indicated by a green checkmark) and a duration of 18s. The most recent run is for the commit `6f62051` pushed by `lwc1116`.

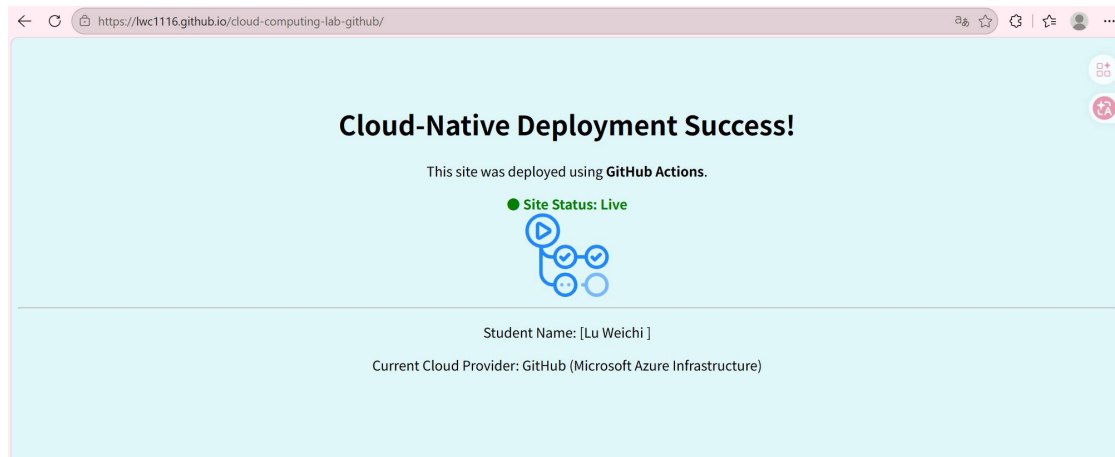
Workflow	Event	Status	Branch	Actor
Update index.html	Cloud-Deployment-Pipeline #3: Commit 6f62051 pushed by lwc1116	Completed	main	lwc1116
Update index.html	Cloud-Deployment-Pipeline #2: Commit 9108311 pushed by lwc1116	Completed	main	lwc1116
Update index.html	Cloud-Deployment-Pipeline #1: Commit 9c8af45 pushed by lwc1116	Completed	main	lwc1116

Action:

Commit and wait for the Actions workflow to finish.

Refresh your live website – the GitHub Actions logo should appear.

Screenshot 12 – Website showing the added image



IV. Reflection Questions

1. Where is your website stored?

Answer: The website is stored on GitHub's cloud servers. The HTML file resides in my GitHub repository, and GitHub Pages publishes it through a global CDN (Content Delivery Network). No local storage is involved.

2. Did you have to install any software?

Answer: No. The entire experiment was performed inside a web browser (Chrome/Edge/Firefox). This demonstrates a key benefit of cloud computing – zero local installation, only an internet connection is needed.

3. Problems Encountered and Proposed Solutions

Problem	Cause	Solution
When I first clicked the Actions tab, workflows were disabled.	GitHub disables Actions automatically on forked repositories for security reasons.	Clicked the “I understand my workflows, go ahead and enable them” button, then made an extra commit to trigger the pipeline.
I saw “File could not be edited” when trying to commit CSS changes.	Likely a browser cache or temporary GitHub session issue.	Refreshed the page, copied my code, pasted it again, and retried the commit – it worked.

V. Conclusion

In this experiment, I successfully forked a GitHub repository, enabled GitHub Pages with GitHub Actions, and deployed a live website automatically after each code change. I also customized the site's appearance (background color) and added an external

image, witnessing how cloud-based CI/CD pipelines work. No local software was required, highlighting the power of PaaS and cloud automation. The experience reinforced key cloud concepts: on-demand computing, continuous deployment, and global accessibility through a managed platform.

Additionally:

I used QQ's screenshot feature and sent the screenshots taken during the experiment to myself to save them. Through these timestamps, I hope to show you, teacher, that I did the experiment myself.

